

QA LEADERSHIP ACADEMY

QA Career Progression Framework

A clear ladder describing what growth looks like at each level — so promotion is earned against evidence, not politics.

Purpose

A progression framework describes, level by level, what is expected of a QA engineer — in scope, autonomy, technical depth and influence. It answers the two questions every good engineer eventually asks: "what does the next level actually require of me?" and "how is that judged fairly?" Without one, promotion becomes a matter of who shouts loudest or who happens to be visible to the right manager.

When to use it

- When a team has no ladder at all — as at Northstar, where there is no QA career framework.
- To anchor development plans (this framework defines the "next level" an IDP aims at).
- In promotion cases, to argue from evidence against a shared bar rather than sentiment.
- In hiring, to pitch the level of a role accurately and set fair expectations.

Instructions

1. Adopt the levels below or adapt the labels to match your organisation's existing bands.
2. For each level, describe behaviour — what the person does — not years of experience.
3. Make each level a superset of the one below: higher levels widen scope and influence, they do not just "do more testing".
4. Separate the individual-contributor track (towards test architecture) from the management track (towards leading people). Neither is superior.
5. Calibrate real people against it as a group, so the bar means the same across the team.

Worked example

A four-rung ladder, calibrated against Northstar's team.

Level	Scope	Autonomy	Technical	Influence
QA Engineer	Own tasks within a story	Works with guidance; asks early	Executes tests; learning design techniques and tooling	Learns from the team

Level	Scope	Autonomy	Technical	Influence
(Junior)				
QA Engineer	Owens testing for a feature	Independent on routine work	Solid test design; contributes to automation; competent in one specialism	Trusted within the squad
Senior QA Engineer	Owens quality for a squad or a cross-cutting capability	Fully autonomous; handles ambiguity	Deep in at least one area (e.g. automation, performance); sets local standards	Shapes how the squad works; mentors others
Lead / Principal QA	Owens strategy across squads or a discipline	Sets direction; trusted with the hardest problems	Architect-level in a discipline; defines org-wide approach	Influences engineering leadership; grows other seniors

The team, calibrated against the ladder

Engineer	Current level	Evidence	Next-level focus
Emma Clarke	QA Engineer (Junior)	Still ramping; works with guidance across the board.	Reach Working level on core competencies; own a feature end-to-end.
Grace Okonkwo	QA Engineer	Independent on web testing; growing accessibility depth.	Deepen one specialism to Proficient to progress toward Senior.
Hassan Malik	QA Engineer	Reliable on mobile; solid exploratory work.	Broaden from execution into test design leadership on the squad.
Dan Whitmore	Senior QA Engineer (technical track)	Deep, autonomous on automation and CI/CD.	Move from doing to enabling — raise the whole team's automation; toward Principal.
Ravi Kapoor	Senior QA Engineer	Autonomous; Expert-level payments domain; sets local standards.	Convert domain depth into cross-squad influence and documentation.
Sofia Alvarez	Senior QA Engineer, on the Lead track	Owens quality thinking beyond her squad; already the informal lead and mentor.	Formalise strategy and stakeholder influence to step up to Lead.

Note that the ladder holds two distinct senior shapes — Dan on a technical/architecture path and Sofia on a leadership path — at the same level. That is deliberate: not everyone grows by managing people.

How to interpret the results

- Level is defined by the scope of problem someone owns, not by how many tests they write. A brilliant executor who never widens their scope is a strong QA Engineer, not a Senior one.
- People can sit between rungs; the framework names the direction, not a precise coordinate. Use it to describe the gap, then let the IDP close it.
- If most of the team clusters at one level, either your bar is wrong or your hiring is one-note — both worth investigating.
- Promotion should follow demonstrated behaviour at the next level, not a promise to start behaving that way once promoted.

Blank reusable version

Level	Scope	Autonomy	Technical	Influence

Calibration grid — one row per engineer:

Engineer	Current level	Evidence	Next-level focus

Common mistakes

COMMON MISTAKES

Defining levels by years of experience rather than behaviour. Forcing every senior person onto a management track, which pushes strong technical people out the door. Promoting on potential ("they'll grow into it") and then wondering why the person is struggling. Writing a framework so abstract that two managers reading it would rate the same person differently. Treating the ladder as a one-way ratchet where nobody is ever allowed to be happy and excellent at their current level.

INSIDE STLC PRO TIP

The clearest sign your framework works is that an engineer can read it and self-diagnose their own gap to the next level without you in the room. If they need you to interpret it, it is too vague — and vagueness is where bias and favouritism live.